



你把正文给我

当然，下面是正文。

Code-as-Knowledge 认知引擎产品设计说明书 (v2.1)

1. 产品愿景与设计哲学

1.1 愿景

构建一个以代码即知识（Code-as-Knowledge）为核心的认知引擎，将复杂长文本世界编译为可执行、可验证、可回溯的统一代码图谱（Unified Code-Graph），成为超级长文本理解与多跳推理的基础设施。

1.2 核心哲学

1. 代码即最高维知识载体：自然语言只出现一次，用于输入；系统内部的“知识形态”以 Python Class 实例与其关系图为唯一真理源。
2. 一次结构化，终身确定性：文本只允许经历一次从 Text→结构化 JSON→Code IR 的结构化过程，之后所有转换（JSON→Code→AST→Graph）必须是确定性的、可重放的程序逻辑。
3. 零二次语义损耗：推理阶段禁止出现 Code→Text→Code 的往返翻译，Agent 只通过原生 Python API 在 Code-Graph 上进行高效遍历与计算，信息损耗趋近于 0。
4. 逃生舱机制：每一个结构化断言都绑定原始文本证据（Evidence），在遇到模糊语义或高风险决策场景时，系统可以随时“解压回到原文”，由人类或高权限 Agent 进行终审。
5. 减熵架构：系统刻意压制一切不必要的中间态（多级摘要、重复 embedding 管线等），在保证安全与可验证的前提下保持架构极简，降低整体语义熵增风险。

2. 整体架构设计

2.1 高层模块

系统围绕唯一真理源 Unified Code-Graph 构建四个核心支柱模块：

1. 声明式本体层（Ontology as Code）：使用 Python Dataclass / Pydantic 定义世界内可识别的实体、事件、关系与内置业务逻辑方法，构成“认知物理规则”。
2. 知识编译器（LLM Knowledge Compiler）：负责将原始自然语言文本一次性结构化为符合本体 Schema 的 JSON，并自动转化为 Python Class 实例化代码（Code IR）。

3. 认知防火墙 (Cognition Firewall & AST) : 使用 AST、类型系统与本体内嵌逻辑校验, 拦截一切语法、类型、引用和业务逻辑层面的错误知识写入。
4. Agent 沙箱推理环境 (Python Execution Sandbox) : Agent Planner 只在沙箱中编写、运行 Python 查询脚本, 通过原生 API 与 Code-Graph 交互, 完成多跳推理与证据解压。

2.2 数据流转 (Text→Code→Graph)

1. 文本摄入 : 输入为长篇原始自然语言文本 (如法律条文、小说章节、史料、业务规范等)。
2. 单次结构化 :
 - LLM 读取文本与本体 Schema 说明, 仅被授权输出严格对齐 Schema 的 JSON ;
 - 使用 Pydantic / Instructor 等工具进行强约束与自动纠错, 确保 JSON 结构完整且类型正确。
3. Code IR 生成 : 系统将 JSON 确定性转换为 Python Class 实例化代码片段 (Code IR) , 不再额外引入新的自然语言解释层。
4. AST 与静态检查 :
 - 使用 `ast.parse()` 确认代码可编译 ;
 - 使用类型检查 (Mypy / 反射) 与引用完整性检查, 确保不存在空指针或指向虚无实体的引用。
5. 业务逻辑一致性检查 : 调用类内部硬编码的逻辑校验方法 (例如时间线冲突检测、实体关系不兼容判断) , 从领域角度过滤掉自相矛盾的知识声明。
6. 写入 Unified Code-Graph : 所有通过三重防线的 Code 对象被增量合并进 Code-Graph , 成为后续推理的唯一事实源。

2.3 减熵约束准则

1. 禁止多级摘要管线 : 在 Text→JSON→Code 的知识编译瓶颈中, 禁止引入“先摘要、再结构化”的中间 Text 层, 避免对原文进行重复有损压缩。
2. 检索与编译分离 : embedding / RAG 检索仅用于“找到哪一段 Text 需要被编译”, 不参与“如何解释该段 Text”。解释只在 LLM→JSON 这一环发生一次。
3. 禁止 Code→Text→Code 回路 : 系统明确禁止任何“先将 Code/执行结果转为自然语言, 再喂给 LLM 生成新 Code”的闭环。所有增量 Code 必须直接基于 Text 证据与本体 Schema 生成。

3. 本体层与数据模型设计

3.1 设计原则

1. 胖模型 (Rich Domain Model) : 将尽可能多的业务规则下沉至类内部方法中, 让对象自身具备自校验和行为能力, 而不是依赖外部 Prompt 推理。
2. 证据优先 : 每个结构化字段与关系声明都必须附带至少一个 Evidence, 便于在推理边界处一键回到原文。

3. 类型安全与可演化性：所有模型基于 Pydantic / BaseModel 构建，可随着业务演进增量扩展字段而不破坏历史对象。

3.2 核心数据结构（伪代码）

```
from pydantic import BaseModel, Field
from typing import List, Dict, Any, Optional

class Evidence(BaseModel):
    """逃生舱机制：固化原始文本，对冲第一次翻译损失"""
    source_text_block: str
    source_span: List[int]
    confidence: float
    timestamp: float

class KnowledgeNode(BaseModel):
    """所有认知图谱节点的基类"""
    node_id: str
    node_type: str
    evidences: List[Evidence] = Field(default_factory=list)

    def get_detailed_context(self) -> str:
        """一键回溯至无损原始文本"""
        return "\n--\n".join([e.source_text_block for e in self.evidences])

class CharacterNode(KnowledgeNode):
    """角色/实体节点：包含自治的行为与属性推导方法"""
    name: str
    properties: Dict[str, Any]

    def get_participated_events(self, graph_context: Any) -> List[Any]:
        """通过图上下文查找该角色参与的事件"""
        return graph_context.get_edges_by_source(self.node_id, edge_type="PARTICIPATE")

class EventNode(KnowledgeNode):
    """事件节点：内嵌时间线与逻辑一致性校验规则"""
    title: str
    happened_time: str
    participants: List[str]

    def check_timeline_contradiction(self, other_event: "EventNode") -> Optional[str]:
        """示例：时间线冲突检测"""
        if self.happened_time == other_event.happened_time and self.node_id != other_event.node_id:
            return f"逻辑冲突：实体同时参与了不兼容的事件 [{self.title}] 与 [{other_event.title}]"
        return None
```

3.3 关系与图结构

1. 关系建模：关系可显式建模为 RelationEdge 对象，包含 source_id、target_id、edge_type、evidences 等字段，用于表达参与、因果、依赖等关系。
2. 图存储：Unified Code-Graph 可以先实现为内存图（基于字典和列表），后续再演进为图数据库（如 Neo4j、TigerGraph）或自研内存索引层，但对上层 API 保持不变。

3. 图 API 约定：对 Agent 暴露的 API 需尽量简洁，例如：`get_node(node_id)`、`get_edges_by_source(source_id, edge_type)`、`find_events_by_time_range(time_range)` 等。

4. 知识编译器设计 (LLM as Compiler, not Writer)

4.1 职责边界

1. 唯一职责：Text→JSON 结构化：LLM 只负责把原始文本解读为符合本体 Schema 的 JSON，不直接输出自然语言解释或业务判断。
2. 禁止职责：Code→Text→Code：
 - LLM 不被授权对已有 Code 进行自然语言解释后再生成新 Code；
 - 新的结构化知识只能来源于原始 Text 与其 Evidence，而不是已有 Code 的“转述版”。
3. 自我修正模式：当 AST / Pydantic 校验失败时，系统将错误日志（结构化提示）反馈给同一个 LLM 会话，让其在“编译失败”上下文中修正 JSON，而非重新生成自然语言描述。

4.2 编译流程

1. 输入：
 - 原始文本片段；
 - 当前本体 Schema 说明；
 - 可选的上文 Code-Graph 上下文摘要（例如已有节点 ID 列表）。
2. 输出：完全符合 Schema 的 JSON 对象数组，例如 `CharacterNodePayload[]`、`EventNodePayload[]`。
3. 转换为 Code IR：服务端将 JSON 投射为具体的 Python Class 实例化语句（字符串或 AST），不再依赖 LLM 生成代码字符串。
4. 校验与落库：通过认知防火墙三道关卡后，写入 Unified Code-Graph。

5. 认知防火墙与验证引擎

5.1 三道防线

1. AST 静态语法门禁 (Syntactic Gate)：
 - 使用 `ast.parse()` 对所有 Code IR 进行语法检查；
 - 任何语法错误（缩进、括号、非法标识符等）直接拒绝进入图谱，并返回错误日志给知识编译器修正。
2. 类型与引用完整性验证 (Type & Reference Check)：
 - 使用 Mypy / 反射确保类型声明与本体定义一致；
 - 检查所有 Relation 边的 `source_id`、`target_id` 是否指向已存在或当前批次即将创建的合法节点，杜绝悬挂引用 (dangling reference)。

3. 胖模型业务一致性消解 (Logical Consistency) :

- 在写入图谱前，对新对象执行其内置校验方法（如 `check_timeline_contradiction`）；
- 若检测到冲突，则打包冲突对象及其 Evidence，触发人工介入或高级仲裁 Agent 做逻辑平差，不直接写入图谱。

5.2 人工与高级 Agent 仲裁

1. 冲突包展示：UI 或仲裁 Agent 会看到：

- 冲突对象的代码 diff；
- 相关 Evidence 原文片段；
- 触发的校验规则说明。

2. 决策模式：人类或高级 Agent 可以选择：

- 接受新知识并修改旧知识；
- 拒绝新知识；
- 分叉多个世界观版本（例如“版本化图谱”），用于假设推理。

6. Agent 推理循环与沙箱执行

6.1 推理闭环设计

当用户抛出复杂问题（例如“根据第 3、5、9 章的所有线索判断某角色是否具备不在场证明”）时，系统执行以下闭环：

1. 用户查询 → Agent Planner：

- Planner 读取用户自然语言问题；
- 分解为一系列对 Code-Graph 的查询与计算任务。

2. Planner 生成 Python 查询脚本：

- 只根据本体层 API 与图 API 生成 Python 代码；
- 不插入中间自然语言推理链，避免 Code→Text→Code。

3. 沙箱执行：

- 在隔离的 Python 沙箱中运行脚本，调用 `.get_node()`、`.get_participated_events()`、`.check_timeline_contradiction()` 等方法，对图进行多跳遍历与过滤；
- 此阶段信息损失为 0，因为所有运算都在结构化 Code-Graph 上进行。

4. 触发逃生舱：

- 当脚本发现某些节点的结构化字段不足以支撑决策（例如角色动机、细腻情绪），通过 `node.get_detailed_context()` 解压原文；
- Planner 将这部分原文作为“证据文本”融入最终回答，而不是作为再次生成代码的依据。

5. 终审合流：

- Agent 将“确定的逻辑拓扑结构”与“局部无损证据文本”结合，给出最终结论；
- 如仍存在高不确定性，可显式标注“需要人工审核”并打包证据链。

6.2 Agent 行为约束

1. 只写 Python，不写 Prompt 链：Agent Planner 在推理阶段只被授权编写 Python 脚本，不构造多轮自然语言 Prompt 作为推理载体。
2. 禁止 Code→Text→Code 循环：不允许出现“解释代码→再让 LLM 生成新代码”的模式，所有新知识必须回到 Text→JSON→Code 的编译链路。
3. 模糊场景的唯一合法路径：当遇到模糊语义时，只能：Evidence 指针 → 解压原文 → 交由人类或高级仲裁 Agent，而不是直接让 LLM 在此基础上生成新 Code。

7. MVP 实施路径

7.1 Phase 1：硬编码本体与纯代码推理机

目标：在完全不接入 LLM 的前提下，验证 Code-Graph + 沙箱执行的闭环可行性。

1. 使用 Pydantic 定义核心胖模型类：Evidence、KnowledgeNode、CharacterNode、EventNode 等。
2. 手工将数篇短文（如短篇小说片段、案例材料）翻译为 Python 实例代码，构建内存 Code-Graph。
3. 实现基础图 API 与沙箱执行环境，让 Agent（可以是人类开发者）通过 Python 脚本在图上完成多跳查询与简单逻辑推理。
4. 验证指标：
 - API 易用性：人类 Agent 能在几分钟内写出有效查询脚本；
 - 推理可解释性：所有结论都能追溯到明确的节点与 Evidence。

7.2 Phase 2：接入 LLM，打通 Text→JSON→Code

目标：让 LLM 在严格约束下参与知识构建，但不改变 Phase 1 核心架构。

1. 引入 LLM + Instructor / Pydantic JSON 输出约束，搭建 Text→JSON 编译服务。
2. 开发 JSON→Code IR 转换器，将 JSON 自动转为 Python Class 实例化代码。
3. 集成 AST / 类型 / 逻辑三重校验，形成完整认知防火墙。
4. 验证指标：
 - 自动转换通过率 ≥ 90%（即绝大多数 JSON 可以一次性通过校验写入图谱）；
 - 失败样例可被 LLM 在错误日志提示下自动修复大部分。

7.3 Phase 3：丰富内置方法与逃生舱驱动推理

目标：将更多复杂业务逻辑下沉至本体类内部，形成高维度、低信息损失的推理闭环。

1. 为核心节点类型设计更丰富的内置方法，例如：
 - 时间线推理（前后因果、重叠冲突）；
 - 社交网络推理（间接关系链条长度、影响力测度）；
 - 规则约束（合规、权限、业务规则）。
2. 在 Agent 推理脚本模板中显式加入逃生舱调用逻辑，使其在关键节点自动解压证据文本。
3. 探索“版本化 Code-Graph”以支持假设推理和多世界观并存。
4. 验证指标：
 - 在长文本、多章节、多角色场景下，能稳定完成高跳数推理；
 - 对比传统 RAG，在全局逻辑一致性任务上显著提升正确率与可解释性。

8. 与现有 AI Coding / RAG 系统的差异

1. 不把 LLM 当通用写代码助手，而是当一次性的结构化编译器：彻底避免“LLM 无约束写代码”导致的脆弱性，将其输出限制在 JSON，代码由确定性程序生成。
2. 推理载体从 Prompt 变为 Code：传统 Agent 在 Prompt 中“口头讨论”推理链，本系统让 Agent 在 Python 中“显式编程”，逻辑链条透明且可重放。
3. 知识表示从三元组变为胖模型 Code：相比死板三元组，本系统通过带行为的对象（Class + Method）表达更复杂的业务规则和自洽性约束。
4. 语义逃生舱与原文对照：所有结构化知识都可以回溯到原始文本片段，避免被多层摘要与向量化掩盖细节。

9. 后续演进方向（可选）

1. 支持多语言与多模态 Evidence：将 Evidence 扩展为支持多语言文本、表格、图像甚至代码片段，为更多样的知识源提供统一接口。
2. 引入形式化验证：对关键领域（如金融合约、合规规则）引入轻量的形式化验证工具，进一步加强逻辑一致性。
3. 生态开放：为第三方开发者暴露构建本体、扩展节点类型与挂载自定义验证逻辑的插件机制。

本文档是基于“代码即知识”、“一次结构化、零二次损耗”与“禁止 Code→Text→Code 闭环”三大原则，结合你当前的 2.0 设计回答重新整理的 v2.1 版产品设计说明书骨架。